6.1 Introduction

This document presents the architecture and detailed design for the software for the Chess Learning Engine project. The Chess Learning Engine is a windows based, self-learning artificial intelligence designed to play chess and learn along with the player. The engine will implement an approximate Q-learning algorithm to learn chess using the given ruleset. This document highlights the project's high-level structure, the major software components, and the interactions between them.

6.1.1 System Objectives

The objective of this system is to provide a window-based chess application to explore the application and integration of self-learning engine that can play, analyze and improve its strategic performance with minimal user intervention. The system shall integrate a graphical user interface for better user experience. The system shall use reinforcement learning to recognize board states and calculate optimal moves. The game states and piece positions will be identified through a core logic engine and translated into weights for the neural network. The system will then use a reinforcement learning algorithm to adjust its internal evaluation parameters, ensuring that the system continues to improve its performance.

6.1.2 Hardware Interface

6.1.2.1 Mouse – A standard optical or trackpad pointing device is required to select pieces and click menu.

6.1.2.2 Processor – A minimum processor of 2.0 GHz is recommended for the system to optimally manage the core processing.

6.1.2.3 Memory – The system requires a minimum of 1GB of memory space for its move-search process.

6.1.2.4 Storage – The application requires 1GB of local disk space to store the game files and learning database.

6.1.2.5 Display – The system is designed to work with a display device with 1920x1080 and 2560 x 1440 resolution.

6.1.3 Software Interface

6.1.3.1 Python – The system is built using python 3.12 as its primary language.

6.1.3.2 PyTorch – The system will use the Pytorch library which provides the machine learning framework for the engine's neural network.

6.1.3.3 Numpy – This library will support the high-performance mathematical calculation and board-state representation as numerical arrays needed for the logic engine.

6.1.3.4 Pygame – This python library will provide the graphical rendering engine and event handling for the GUI

6.1.3.5 Development Environment Interface – Development will be done using Visual Studio Code, with GitHub repo for version control.

6.1.4 Human Interface

6.1.4.1 Graphical User Interface (GUI) – The GUI will provide a 2D representation of an 8x8 chess grid. The user will interact with the chess board and piece through mouse clicks by selecting a piece and selecting a highlighted tile. The GUI will have buttons to start new game, save game, and exit. The GUI will have a move history panel on the right side of the chess board, below the buttons.

6.2 Architectural Design

6.2.1 Major Software Components

The Chess Engine is decomposed into several major subsystems to ensure that functional requirements are itemized and proven.

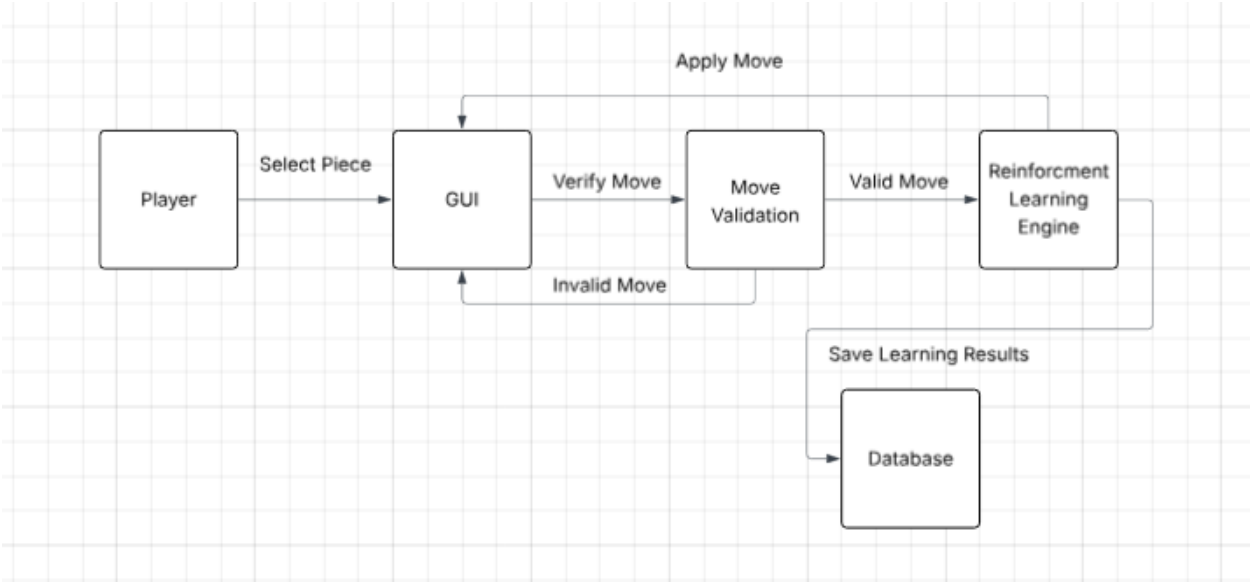
- User Interface Subsystem: Directly corresponds to functional requirements for board rendering and user interaction. It handles the visual 8x8 grid and accepts input from mouse and keyboard.
- Move Validation Subsystem: Implements the core logic required to enforce FIDE rules. It ensures only legal moves are processed.
- Reinforcement Learning Subsystem: Contains the neural network and training logic. This subsystem is responsible for calculating board states and updating its knowledge base after games.
- Data Management Subsystem: This subsystem is responsible for the persistent storage of the learning database and saving/loading game files.

6.2.2 Major Software Interactions

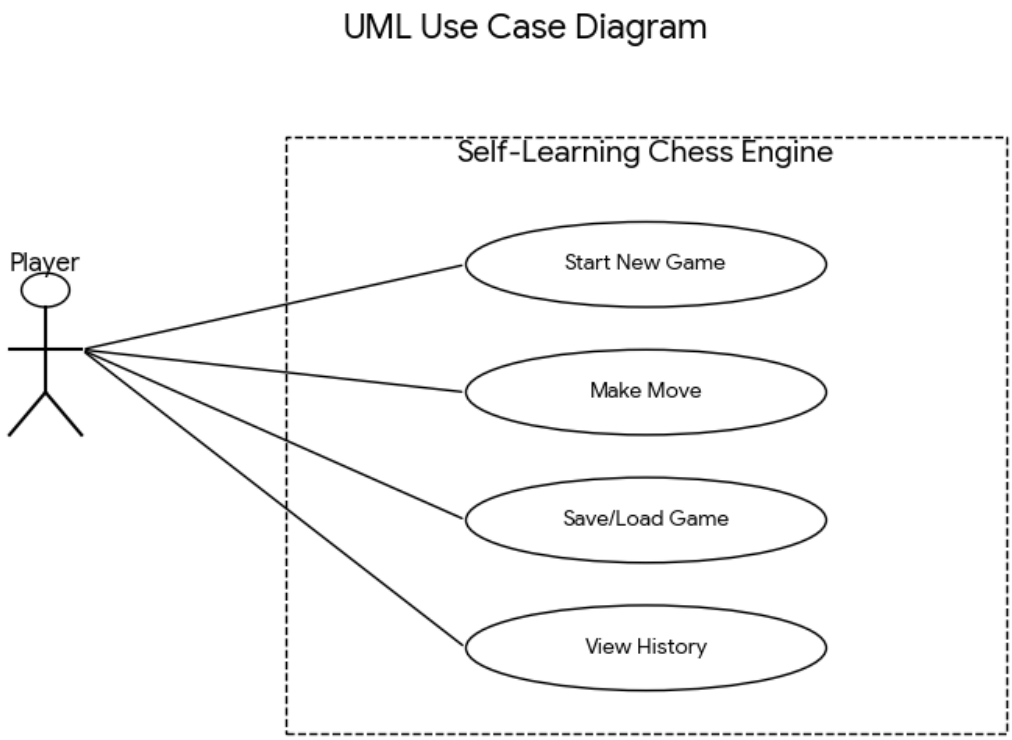
- GUI to Move Validator: When a user attempts a move, the user interface sends the coordinates to the move validation subsystem via a direct function call. The move validation subsystem then returns a Boolean signal indicating if the move is valid.
- Move Validator to Engine: Once a move is confirmed valid, the updated board state is passed to the reinforcement learning subsystem. The reinforcement learning subsystem processes the vector-based representation of the board to generate a response move.
- Engine to Learning Database: At the conclusion of a match, the reinforcement learning subsystem sends the final game outcome (win/loss/draw) and the move history to the learning database.

6.2.3 Architectural Design Diagrams

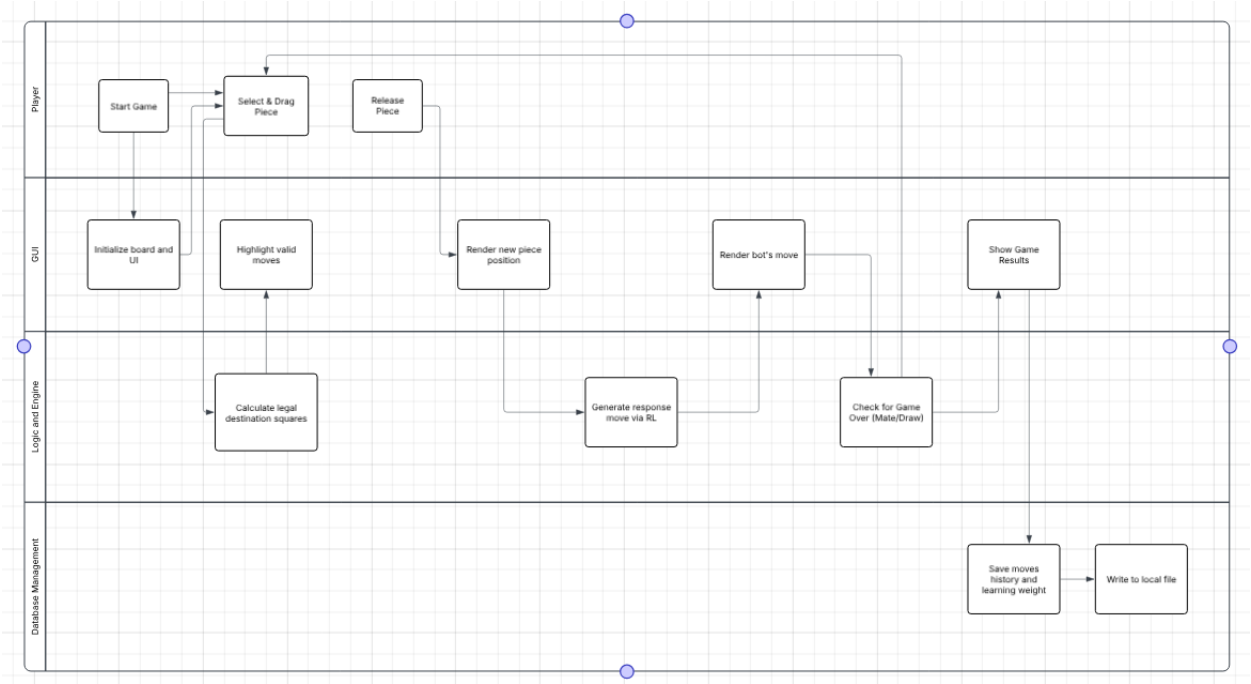
Components/ Data flow Diagram



Use Case Diagram



Swim Lane Diagram.


Player
GUI
Logic and Engine
Database Management
Start Game
Select & Drag Piece
Release Piece
Initialize board and UI
Highlight valid moves
Render new piece position
Render bot's move
Show Game Results
Calculate legal destination squares
Generate response move via RL
Check for Game Over (Mate/Draw)
Save moves history and learning weight
Write to local file